BookMate — 50 Interview Questions, Answered (Fresher Level)

How to use this: each answer is written the way you'd *say* it out loud — 2 to 6 sentences. The → line is the extra detail to add only if the interviewer pushes. Numbers in here are the real ones from the codebase (`app/config.py`, `app/retrieval/firewall.py`, `app/store/vectors.py`), so you can defend them.

The one sentence everything hangs off: chunks are stored in reading order, so "everything I've read" is a contiguous prefix `E[:n_p]` of a NumPy array — the spoiler rule is enforced by which rows exist in the array being searched, not by a filter or a prompt instruction.

A. Project Understanding

1. Tell me about your BookMate project.

BookMate is a chatbot you can ask questions about a book you're in the middle of reading, and it will never spoil what's ahead. You tell it your reading position, and it answers only from the part you've already read. Technically it's a RAG system: the book is chunked, embedded, and stored in reading order, and retrieval searches only the slice of the embedding matrix that corresponds to pages behind your position. I built it because the interesting problem wasn't "chat with a book" — it was making the safety property structural instead of asking the model politely not to spoil.

> **If pushed:** Stack: Python, FastAPI, SQLite + FTS5, NumPy, fastembed (bge-small), LangGraph for the agent flow, React front end with a position slider.

2. What problem does BookMate solve?

If you're halfway through a novel and confused — "wait, who is this character, why did she leave?" — you can't Google it and you can't ask ChatGPT, because both will tell you the ending. BookMate answers that question using only the chapters you've actually read.

3. Why did you build a spoiler-safe reading companion?

Because it's a real need with a clean technical shape. "Don't spoil" is a constraint you can express exactly — as a position in the text — which makes it a good problem: I could turn a fuzzy product rule into a hard invariant and then write tests that prove it holds.

4. What is RAG, and why did you use it?

RAG is Retrieval-Augmented Generation: instead of relying on what the model memorised during training, you retrieve relevant passages from your own data and put them in the prompt, and the model answers from those. I used it for two reasons — the model doesn't know an arbitrary user-uploaded book, and more importantly retrieval is the layer where I can enforce the spoiler boundary. Whatever I don't retrieve, the model cannot say.

5. Explain the complete architecture of BookMate.

Two phases. **Ingest (offline):** download or upload the book → clean it → split into chapters → chunk into 320-word overlapping chunks in reading order → embed each chunk → store the text and word offsets in SQLite and the embeddings as one NumPy matrix per book, row *i* = chunk *i*. **Query (online):** the API takes `{question, book_id, position}` → a LangGraph pipeline runs: classify the question → retrieve within the boundary → check the evidence is enough → answer with citations → validate that the citations are real → return. If evidence isn't enough, a branch asks an air-gapped "oracle" whether the answer exists later in the book, and picks which kind of refusal to show.

> **If pushed:** Two stores: Store A is the chunks behind the reader (the answer path can see it); Store B is the full book, reachable only by `app/retrieval/oracle.py`, which returns a single boolean and never text.

6. Walk me through what happens when a user asks a question.

1. The request arrives with the question and a reading position (a word offset). 2. I compute `n_p` = how many chunks lie entirely behind that position — `SELECT COUNT(*) FROM chunks WHERE word_end <= position`. 3. Retrieval: I embed the question, and multiply it against `E[:n_p]`, the first `n_p` rows of the embedding matrix, to get cosine scores; in parallel a keyword search runs over the same range. 4. The two ranked lists are fused with RRF, I take the top 5, and pull each hit's immediate neighbours for context. 5. Those passages go into the answer prompt with their IDs. 6. The model answers and cites passage IDs, or replies `INSUFFICIENT`. 7. A validator checks every cited ID was actually supplied; if not, retry up to twice. 8. The answer, its citations and the outcome go back to the UI.

7. How do you ensure the chatbot doesn't reveal spoilers?

The unread part of the book is never in the data being searched. Because chunks are stored in narrative order, "already read" is the array slice `E[:n_p]`, and a similarity search over that slice literally cannot return row `n_p` — that row isn't in the operand. So there's no filter that someone could forget to apply on the vector path. On the keyword path, which is SQL, there *is* a condition (`c.idx < :n_p`), and I have a mutation test that deletes that line and proves the test suite goes red.

> **If pushed:** Plus a final `assert_safe()` before anything reaches a prompt: if any chunk has `word_end > position` it raises `FirewallBreach` instead of filtering, because a silent filter would hide the exact bug the system exists to prevent.

8. What do you mean by "temporal RAG"?

Normal RAG retrieves from the whole corpus. Temporal RAG adds a time dimension: each chunk has a position in the narrative, and every query carries a "as of when" bound, so retrieval is over a moving window of the corpus rather than all of it. It's the same idea as an as-of query in a database — answer using only what was visible at that point.

9. What is the "reading boundary" in your system?

It's a word offset `P` — the reader's exact position in the book. Everything with `word_end <= P` is allowed; everything past is not. I use `word_end`, not `word_start`, because chunks overlap by 40 words, so a chunk that starts before the reader can still contain words they haven't reached. Requiring the whole chunk to be behind the reader is conservative by one chunk and keeps the eligible set an exact prefix.

10. How do you know which pages the user has already read?

The user tells us — in the UI it's a slider for the page they're on. Pages are a synthetic view: 250 words per page, so page → word offset is just multiplication. Word offset is the ground truth because it's stable and exact; page numbers vary by edition.

B. RAG Fundamentals

11. What is Retrieval-Augmented Generation (RAG)?

A pattern where, before the model answers, you fetch relevant text from an external knowledge source and put it in the prompt. The model's job becomes reading and summarising supplied evidence rather than recalling facts. It gives you fresh/private data, citations, and control over what the model can possibly say.

12. Why use RAG instead of simply asking an LLM the question?

Three reasons. The model may never have seen this book — a user upload definitely isn't in training data. Without evidence models hallucinate plausible details. And for my project specifically, RAG is the only design where I *control* what the model sees; if the ending isn't in the prompt, it can't be quoted back.

13. What are the main components of a RAG system?

Ingestion (load, clean, chunk), an embedding model, a vector store, a retriever (usually a similarity search, often plus keyword search), a prompt that packages the retrieved passages, and the generator LLM. In mine there are two extras: a firewall that bounds retrieval, and a validator that checks the answer only cites passages that were actually supplied.

14. Explain the retrieval process in your project.

I embed the question with the same model used for the chunks, then take the dot product against `E[:n_p]` — because every vector is L2-normalised, that dot product *is* cosine similarity — and take the top-scoring rows. Separately I run a SQLite FTS5 keyword search over the same chunk range. The two ranked lists are fused with Reciprocal Rank Fusion, I take the top 5, expand each hit to its neighbours (idx±1), and sort the final passages in narrative order before putting them in the prompt.

> **If pushed:** Narrative order, not relevance order, because a model reading passages in the order the reader met them makes much better sense of them.

15. What is an embedding?

A list of numbers — a vector — that represents the meaning of a piece of text. Texts with similar meaning get vectors that point in similar directions. Mine are 384 numbers per chunk, from `BAAI/bge-small-en-v1.5`.

16. Why do you generate embeddings for book passages?

So I can search by meaning instead of by exact words. A reader asks "why did she leave?" and the book says "she departed without a word" — no shared keywords, but the embeddings are close, so the passage is found.

17. What is a vector embedding?

Same thing as an embedding: a fixed-length numeric vector in a high- dimensional space where distance encodes semantic similarity. Fixed-length is the useful part — every chunk, long or short, becomes 384 numbers, so they're all directly comparable.

18. What does semantic similarity mean?

How close two pieces of text are in *meaning*, not in wording. "The dog is happy" and "the puppy is joyful" share almost no words but are semantically almost identical, and their embeddings reflect that.

19. How does semantic search work?

Embed every document once and store the vectors. At query time, embed the query with the same model, compute similarity (cosine) against the stored vectors, sort, take the top K. In my case that's one matrix–vector multiply in NumPy: `matrix @ query_vector`.

20. What is the difference between keyword search and semantic search?

Keyword search matches literal tokens — fast, exact, great for names and rare terms, but blind to synonyms. Semantic search matches meaning — handles paraphrase, but can miss an exact proper noun and is fuzzier. I run both and fuse them, so a question naming "Sherlock" gets the keyword hit and a question phrased loosely still gets the semantic one.

21. Why not simply search the book using SQL text queries?

Because readers don't ask questions in the book's words. `LIKE '%why did she leave%'` finds nothing. SQL text search also can't rank by relevance in any meaningful way. I do still use SQL full-text search (FTS5) — as *one* of two paths, not the only one.

22. What happens if the user's question uses completely different words from the book?

That's exactly the case embeddings handle — the vectors are still close, so the right passage ranks high even with zero word overlap. If it's so different that both paths miss, the answer agent replies `INSUFFICIENT` and the user gets an honest "I don't have enough to answer" rather than a guess.

23. How are book pages/chunks converted into embeddings?

At ingest, after chunking, I batch the chunk texts through the embedding model (fastembed running ONNX locally, no API call), get back an (n_chunks × 384) float32 matrix, L2-normalise every row, and save it as a single `.npy` file per book. Row *i* corresponds to chunk index *i* — that ordering is what makes the whole firewall work.

24. What information do you store along with each embedding?

In SQLite, per chunk: the chunk id, book id, `idx` (its position in reading order), the text, chapter number and title, and `word_start`/`word_end` offsets. The offsets are the important ones — they're what the reading boundary is compared against. The vector itself lives in the NumPy matrix, linked purely by row index.

25. How do you retrieve the most relevant passages?

Cosine similarity for the semantic path, BM25 for the keyword path, fused by RRF, top 5 kept, then widened to neighbours. Then a hard safety re-check before the passages are allowed anywhere near a prompt.

C. Spoiler Prevention — the section they will actually dig into

26. How exactly does your system prevent spoilers?

By making the unread book physically absent from the search, rather than filtering it out afterwards. Chunks are stored in reading order, so the read portion is the contiguous prefix `E[:n_p]`. The vector search operand *is* that slice, so future rows can't be returned. The keyword path adds `c.idx < :n_p` inside the same SQL statement as the MATCH. And before any passage reaches a prompt, `assert_safe()` re-checks `word_end <= position` and raises `FirewallBreach` if anything slipped through.

27. Why do you store embeddings in reading order?

Because it turns "the set of chunks this reader may see" into a *slice*. If the rows were in arbitrary order, "already read" would be a scattered set of row indices and I'd need a filter — something a future code path could forget. In reading order it's `matrix[:n_p]`: one operation, nothing to forget.

28. What do you mean by "prefix slice"?

The first `n_p` rows of the matrix — `E[:n_p]` in NumPy. "Prefix" because reading is sequential: if you've read chunk 40 you've read 0 through 39 too, so the allowed set is always an unbroken run from the start.

29. Give me an example of the prefix-slice approach.

Say the book is 500 chunks and the reader is at word 12,000, which is 43 chunks in. `n_p = 43`. The searchable matrix is `E[:43]`, shape (43, 384). I compute `E[:43] @ q` and get 43 scores. Chunks 44, 45, 300 aren't scored at all — they're not in the array. There is no step where I "remove" them, because they were never there.

30. Why search only the already-read portion instead of retrieving from the entire book?

Two reasons. Safety: retrieve-then-filter means the spoiler existed inside my process, one logging statement or one refactor away from a prompt. And quality: if I retrieve top-5 over the whole book and four hits are from the future, filtering leaves me one passage. Searching the prefix gives me five *usable* passages, because top-5 is computed within the allowed set.

31. What happens if the answer exists only on a page the user hasn't read?

Retrieval brings back whatever is closest within the prefix, the answer agent reads it, sees it doesn't actually answer the question, and replies `INSUFFICIENT`. Then the graph consults the oracle to decide whether to say "this gets answered later, come back when you're further in" or "I don't have enough on this."

32. Does your system retrieve the future page and then filter it out?

No — and that's the design's whole point. Filtering after retrieval is the approach I rejected. On the vector path the future rows are never in the operand, so there is nothing to filter.

33. Why is searching only the allowed prefix safer than retrieving everything and filtering afterward?

Because a filter is a line of code that must be correct at every call site, forever. A slice is an invariant enforced by the shape of the data — the unsafe state can't be represented. Concretely, with a filter there's a moment where a spoiler chunk is a live variable in my process; with a slice there isn't. The only place I couldn't make it structural is the SQL keyword path, which is exactly where I put a mutation test.

> **If pushed:** Mutation test = I delete the `c.idx < :n_p` predicate and assert the suite goes red. A firewall test that still passes when you break the firewall proves nothing.

34. What happens if a user asks "What happens to the character on page 200?" while they have only read page 50?

Page 50 → word 12,500, so `n_p` covers about the first 12,500 words, and the search runs only over that. Nothing about page 200 can be retrieved. The answer agent sees no supporting evidence and returns `INSUFFICIENT`; the oracle — which can see the whole book but returns only a boolean — says "yes, this is resolved later", so the user gets: "I can't answer that without spoiling; ask again once you're further in."

35. What response does BookMate give when evidence isn't available?

One of two refusals: `refuse_spoiler` (it's answered later in the book) or `refuse_thin` (the book doesn't really cover this). They're distinguished internally for evaluation but rendered identically in the UI, because "I can't answer without spoiling" is itself a small leak — it tells you the question *does* get resolved. That's a known, documented limit of about one bit of information.

36. Can an LLM accidentally generate a spoiler even if retrieval is restricted?

Yes — from its own pretrained knowledge. If the book is *Dracula*, the model already knows the ending, and no retrieval firewall can un-train it. I'm explicit about this: the firewall guarantees the model is never *fed* future text, not that it never *knows* it. I mitigate with prompting and citation-forced grounding, and I measure the residual.

37. How do you prevent the LLM from using its own pretrained knowledge?

The prompt says to answer only from the supplied passages and cite them, or reply `INSUFFICIENT`. Then a validator enforces it: every citation in the answer must resolve to an ID that was actually in the evidence, otherwise the answer is rejected and regenerated, up to two retries. An answer that can't be sourced to a supplied passage doesn't ship.

> **If pushed:** And I measure what's left with a "parametric leakage" metric: run the questions with *zero* evidence supplied and count how many the model still gets right. That number is the honest bound on what a firewall can't fix.

38. How do you ensure the generated answer is grounded in retrieved evidence?

Two layers. A deterministic one — parse the citation IDs out of the draft and assert they're a subset of the supplied IDs; this always runs and is load-bearing. And an optional model-based entailment check as a second opinion, which can be switched off because it costs an extra API call and can veto a correct answer.

> **If pushed:** A bug worth mentioning: an empty answer once passed validation, because it made no unsupported claims. Vacuous truth isn't grounding — empty drafts are now rejected explicitly.

39. What happens if the retrieved passages don't contain enough information to answer?

The answer agent says `INSUFFICIENT` and we branch to the oracle. I first tried to detect this cheaply from the similarity score and it didn't work: RRF scores encode rank only, and raw cosine turned out to be corpus-dependent — bge-small scores 0.455 for "what is the capital of Peru?" against Sherlock Holmes, which is not far off a real question's score. So sufficiency is judged where it's actually judgeable: by the model that can read the passages. The score floor survives only as a cheap screen for degenerate cases.

40. How would you test that your spoiler protection actually works?

Four ways, and I built all of them. Unit tests on the boundary (prefix identity, per-position bounds). A shingle leak sweep — take word sequences from future chunks and assert none appear in retrieved text at any position. The mutation test that breaks the SQL predicate and requires the suite to go red. And an end-to-end eval on a synthetic novella with facts planted at known positions, each tagged with a nonce string that provably appears exactly once in the book, so I can report prompt leakage as a number — and it must be 0%.

> **If pushed:** The synthetic book matters: on a real novel a "correct" answer might come from the model's memory rather than from retrieval, so you can't tell a leak from recall. My first eval reported a 17% leak that was entirely a ground-truth artifact — the fact was keyed on a first name that appeared 225 times in ordinary filler.

D. Embeddings & Vector Search

41. What is cosine similarity?

The cosine of the angle between two vectors — it measures direction, not length, so a long passage and a short question can be compared fairly. It ranges from -1 to 1; for text embeddings in practice you see roughly 0.3 to 0.9. I L2-normalise every vector when I save it, so cosine similarity is just a dot product, which makes the whole search one NumPy matrix multiply.

42. How do you determine which passages are most relevant to a question?

`scores = E[:n_p] @ q` gives a cosine score per eligible chunk; `np.argsort(scores)[::-1]` gives me the ranking. That's the semantic ranking. It's then fused with the BM25 keyword ranking via RRF, and the top 5 fused hits are what I keep.

> **If pushed:** I keep the raw cosines separately, because RRF scores are rank-only and useless as a relevance signal — the top RRF hit scores the same whether it's on point or unrelated. That mistake is why my first sufficiency gate passed everything.

43. What happens if two passages have similar embeddings?

They score similarly and both tend to get retrieved, which is often redundant — the same scene from two overlapping chunks. In my system that's partly by design, since I deliberately pull in neighbours for context, and I de-dup by chunk id and cap the total evidence at 9,000 characters. If it were a real problem, MMR (maximal marginal relevance) is the standard fix: reward relevance, penalise similarity to what you've already picked.

44. What is chunking, and why is it needed?

Splitting a document into smaller pieces before embedding. It's needed because an embedding is a fixed-size vector: embed a whole 100,000-word book and you get one blurry average of everything, useless for retrieval. Also, models have context limits, and you want to put only the relevant paragraphs in the prompt, not the whole book. For me there's a third reason — chunks are the unit the reading boundary is defined on.

45. How do you decide the chunk size?

Empirically. I started at 200 words with 80 overlap; the retrieved evidence was visibly cutting off mid-conversation, so narrative answers lost context. I moved to 320 words with 40 overlap. I also cut only on sentence boundaries and never across a chapter break, so a passage never starts mid-sentence — evidence that begins "shocking! He deserves…" reads as noise to a model and to a human.

> **If pushed:** I halved the overlap because overlap is redundant compute: at 80 every word gets embedded 1.25 times, and embedding is my CPU high-water mark on a free-tier instance. Neighbour expansion at retrieval time covers the continuity I lost.

46. What happens if chunks are too small?

Each chunk loses context, so a passage can match the query without carrying enough information to answer it, and pronouns lose their referents ("she left" with no idea who "she" is). You also get many more chunks, so more vectors and more index size.

47. What happens if chunks are too large?

The embedding becomes an average of several topics, so it's a weaker match for any specific question — precision drops. You also waste prompt budget: every retrieved chunk carries a lot of irrelevant text, and on a rate-limited free tier that directly costs you passages you could have included.

48. How would you improve retrieval quality if the wrong passages are being retrieved?

In rough order of effort: check the chunking first, since bad boundaries look like bad retrieval; add the keyword path if it isn't there (it fixes proper nouns that embeddings blur); retrieve a larger pool and rerank with a cross-encoder, which scores the query and passage together instead of comparing two independent vectors; try query rewriting or HyDE for vague questions; and try a stronger embedding model. And build a small labelled question→passage set first, so I can tell whether a change actually helped instead of guessing.

49. What is the difference between embedding a question and embedding a document?

The same model, but many modern embedding models are trained asymmetrically and expect a prefix or an input type — a query is short and interrogative, a document is long and declarative, and telling the model which it's looking at improves the match. In my code `embed_texts` takes an `input_type` of `"query"` or `"document"` for exactly that reason. The one thing that must never differ is the model itself: query and document vectors have to live in the same space, which is why changing the embedding model invalidates every stored vector and forces a full re-index.

50. How would you scale BookMate from one book to millions of documents/users?

Today it's honest about its scale: one `.npy` matrix per book, loaded into memory, brute-force scored with NumPy. That's fine — a book is a few thousand chunks, so a full scan is a millisecond or two, and it's what makes the prefix trick possible. To scale I'd change three things. Storage: move to a real vector database (pgvector, Qdrant, Milvus) with the book id as a partition, so each search still touches one book. The boundary: an ANN index can't be sliced the way an array can, so I'd either keep per-book brute force (still cheap) or use the DB's metadata filter on `idx < n_p` — and I'd be clear that this trades my structural guarantee for a predicate one, so the tests around it become load-bearing. Serving: embed on a separate worker or a hosted embedding API, cache query embeddings, and move ingest into a job queue since it's minutes of pegged CPU. Users are easier than documents — reading position is just a per-user row, and the index is shared.

Three things to have ready

Numbers: 320-word chunks, 40-word overlap, 384-dim `bge-small-en-v1.5`, top-K 5, RRF k=60, oracle threshold 0.62, 250 words/page, 9,000-char evidence budget, 35 tests.

Your one strong opinion: "Retrieve-then-filter and structural-prefix look identical when they work. They differ in what happens when someone edits the code six months later. I picked the one where the unsafe state can't be represented, and I mutation-test the one place I couldn't."

A failure you can talk about: provider errors were being caught and returned as `INSUFFICIENT`, so an outage scored as 100% refusal accuracy — a broken system looked like a perfectly safe one. I added a distinct `error` outcome so an outage can never masquerade as correct behaviour. Same lesson as the empty-answer bug: check that your success metric can actually fail.